

Sistema de Búsqueda de Rutas Aéreas entre Provincias del Perú Mediante Algoritmos de Inteligencia Artificial

Hector Yeran Vilca Cari

Escuela Profesional de Ingeniería de Sistemas
Universidad Nacional del Altiplano
Puno, Perú

Roy Omar Yucra Miranda

Escuela Profesional de Ingeniería de Sistemas
Universidad Nacional del Altiplano
Puno, Perú

Resumen—Este trabajo presenta el diseño e implementación de un sistema interactivo para la búsqueda de rutas aéreas entre las 196 provincias del Perú, aplicando siete algoritmos clásicos de Inteligencia Artificial: BFS, DFS, Dijkstra, A*, Greedy, Minimax y TSP con Simulated Annealing. El sistema modela el territorio peruano como un grafo no dirigido con pesos, donde cada nodo representa una provincia georreferenciada y las aristas conectan provincias dentro de un radio de 150 km usando la distancia haversine. La interfaz gráfica, desarrollada en PyQt5 con visualización cartográfica mediante Leaflet y OpenStreetMap, permite comparar el rendimiento de los algoritmos en tiempo real y determinar automáticamente la ruta óptima. Los experimentos sobre la ruta Lima–Puno demuestran que Dijkstra y A* obtienen consistentemente la ruta más corta (876 km en 8 escalas), mientras DFS y Greedy presentan limitaciones características de sus paradigmas de búsqueda.

Index Terms—algoritmos de búsqueda, grafo de provincias, distancia haversine, Dijkstra, A*, TSP, Simulated Annealing, Minimax, PyQt5, Leaflet

I. INTRODUCCIÓN

El problema de búsqueda de rutas óptimas es uno de los más estudiados en Inteligencia Artificial (IA) y tiene aplicaciones directas en logística, telecomunicaciones y planificación de transporte. En el contexto peruano, donde la geografía accidentada dificulta la conectividad terrestre, modelar la red de provincias bajo un esquema de vuelos directos (línea recta entre coordenadas) permite evaluar la eficiencia relativa de distintos algoritmos sin la complejidad de las rutas viales.

El presente trabajo implementa y compara siete algoritmos de búsqueda sobre un grafo de las 196 provincias del Perú¹, cada una georreferenciada con coordenadas latitud/longitud. Las aristas del grafo conectan pares de provincias cuya distancia geodésica (haversine) no supera los 150 km, lo que obliga a los algoritmos a realizar escalas intermedias, emulando una red de vuelos regionales.

Los objetivos específicos son:

- Construir el grafo de provincias con distancias haversine reales.
- Implementar los siete algoritmos en Python 3.12 con módulos independientes.

- Desarrollar una interfaz gráfica interactiva con visualización geoespacial.
- Comparar los algoritmos en términos de distancia recorrida, número de escalas y tiempo de cómputo.

II. MARCO TEÓRICO

II-A. Representación del Problema como Grafo

Un grafo $G = (V, E)$ está compuesto por un conjunto de vértices V y aristas E . En este trabajo, V es el conjunto de 196 provincias peruanas y cada arista $(u, v) \in E$ tiene un peso $w(u, v)$ igual a la distancia haversine entre ambas provincias, siempre que $w(u, v) \leq 150$ km. El resultado es un grafo no dirigido, conexo y ponderado con $|V| = 196$ nodos y un promedio de 15.5 vecinos por nodo.

II-B. Distancia Haversine

La distancia haversine calcula la distancia de círculo mayor entre dos puntos sobre la superficie esférica de la Tierra, sin considerar la altitud:

$$d = 2R \arcsin \left(\sqrt{\sin^2 \frac{\Delta\phi}{2} + \cos \phi_1 \cos \phi_2 \sin^2 \frac{\Delta\lambda}{2}} \right) \quad (1)$$

donde ϕ_1, ϕ_2 son las latitudes, $\Delta\phi = \phi_2 - \phi_1$, $\Delta\lambda = \lambda_2 - \lambda_1$ la diferencia de longitudes, y $R = 6371$ km el radio terrestre medio. Esta métrica es apropiada para modelar rutas aéreas de punto a punto.

II-C. Algoritmos de Búsqueda

II-C1. BFS (Búsqueda en Anchura): BFS explora el grafo nivel por nivel usando una cola FIFO. Garantiza encontrar el camino con menor número de aristas entre origen y destino, pero no necesariamente el de menor costo si los pesos son heterogéneos [1]. Complejidad: $O(|V| + |E|)$.

II-C2. DFS (Búsqueda en Profundidad): DFS explora cada rama hasta su máxima profundidad antes de retroceder. Es completo en grafos finitos pero no garantiza optimalidad. Para este problema se limita a profundidad 15 con poda por costo para evitar explosión combinatoria. Complejidad: $O(|V|^d)$ donde d es la profundidad máxima.

¹ 195 provincias oficiales más la Provincia Constitucional del Callao.

II-C3. Dijkstra: Dijkstra encuentra el camino de menor costo desde un origen a todos los demás nodos mediante una cola de prioridad mínima. Es óptimo en grafos con pesos no negativos [2]. Complejidad: $O((|V| + |E|) \log |V|)$.

II-C4. A* (A-Estrella): A* extiende Dijkstra incorporando una heurística admisible $h(n)$ que estima el costo desde el nodo n hasta el destino, evaluando $f(n) = g(n) + h(n)$, donde $g(n)$ es el costo real acumulado. En este sistema se utiliza la propia distancia haversine al destino como heurística, lo que la hace admisible (nunca sobreestima) [3]. Complejidad: $O(|E|)$ en el mejor caso.

II-C5. Greedy (Vecino Más Cercano): Greedy selecciona en cada paso el vecino no visitado de menor distancia al nodo actual, sin considerar el costo global. Es rápido pero no garantiza la solución óptima y puede quedar atrapado en mínimos locales.

II-C6. Minimax con Poda Alfa-Beta: Minimax es un algoritmo para juegos de dos jugadores adversariales [4]. Adaptado al problema de rutas: el jugador MAX (el viajero) intenta minimizar el costo total, mientras el jugador MIN (adversario) intenta maximizarlo, simulando condiciones desfavorables. La poda alfa-beta elimina ramas que no afectarán la decisión final. Se usa la heurística haversine para ordenar los nodos candidatos. Profundidad máxima: 8.

II-C7. TSP con Simulated Annealing: El Problema del Viajante (TSP) busca la permutación de todas las ciudades que minimiza la distancia total del recorrido, regresando al origen. Es NP-difícil [5], por lo que se usa Simulated Annealing (SA) como metaheurística de aproximación. SA acepta soluciones peores con probabilidad $e^{-\Delta/T}$, donde T es la temperatura actual, permitiendo escapar de mínimos locales [6].

A diferencia de los otros seis algoritmos (que buscan la ruta más corta de A a B), el TSP resuelve un problema diferente: visitar **todas** las provincias exactamente una vez con el menor recorrido total. Por ello se excluye de la comparación directa de eficiencia $A \rightarrow B$.

III. DESARROLLO

III-A. Arquitectura del Sistema

El sistema sigue una arquitectura modular en Python 3.12 con separación clara entre datos, algoritmos e interfaz:

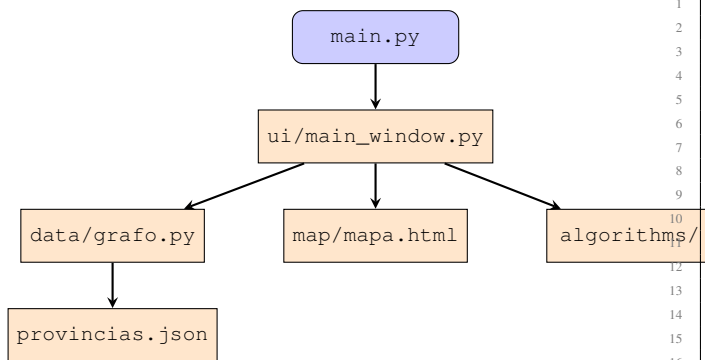


Figura 1. Arquitectura modular del sistema.

El módulo `data/grrafo.py` carga las 196 provincias desde `provincias.json` y construye la matriz de adyacencia filtrando pares con distancia ≤ 150 km. Los algoritmos en `algorithms/` reciben el grafo y los índices de origen/destino, devolviendo el camino y su costo total. La interfaz en `ui/main_window.py` ejecuta los algoritmos en un hilo separado (`QThread`) para no bloquear la UI.

III-B. Construcción del Grafo

El proceso de construcción del grafo se resume en el Algoritmo 1.

Algorithm 1 Construcción del grafo con radio 150 km

Require: Lista de provincias P con coordenadas (lat, lon)

Require: Radio máximo $r = 150$ km

Ensure: Grafo G de adyacencia ponderado

```

1: Inicializar  $G[i] \leftarrow \{\}$  para todo  $i$ 
2: for cada par  $(i, j)$  con  $i \neq j$  do
3:    $d \leftarrow \text{haversine}(P[i], P[j])$ 
4:   if  $d \leq r$  then
5:      $G[i][j] \leftarrow d$ 
6:   end if
7: end for
8: for cada nodo  $i$  con  $|G[i]| = 0$  do
9:   Conectar  $i$  con sus 3 vecinos más cercanos
10: end for
11: return  $G$ 
  
```

El grafo resultante tiene las siguientes características estadísticas:

Cuadro I
ESTADÍSTICAS DEL GRAFO DE PROVINCIAS (RADIO 150 KM)

Métrica	Valor
Nodos (provincias)	196
Radio de conexión	150 km
Vecinos mínimos por nodo	1
Vecinos máximos por nodo	30
Promedio de vecinos	15.5
Nodos aislados	0

III-C. Implementación de Algoritmos Clave

III-C1. Dijkstra:

```

1 import heapq
2
3 def dijkstra(grafo, inicio, fin):
4     dist = {n: float("inf") for n in grafo}
5     dist[inicio] = 0.0
6     previo = {inicio: None}
7     heap = [(0.0, inicio)]
8
9     while heap:
10        costo, nodo = heapq.heappop(heap)
11        if nodo == fin:
12            break
13        if costo > dist[nodo]:
14            continue
15        for vecino, peso in grafo[nodo].items():
16            nuevo = costo + peso
17            if nuevo < dist[vecino]:
18                dist[vecino] = nuevo
  
```

```

19         previo[vecino] = nodo
20         heapq.heappush(heap, (nuevo, vecino)
21     )
22
23     # Reconstruir camino
24     camino, nodo = [], fin
25     while nodo is not None:
26         camino.append(nodo)
27         nodo = previo.get(nodo)
28     return list(reversed(camino)), round(dist[fin],
29         2)

```

Listing 1. Implementación de Dijkstra

III-C2. A* con Heurística Haversine: La función de evaluación es $f(n) = g(n) + h(n)$, donde $h(n)$ es la distancia haversine directa desde n hasta el destino:

```

1 def h(n):
2     pn, pf = provincias[n], provincias[fin]
3     return haversine(pn["lat"], pn["lon"],
4         pf["lat"], pf["lon"])
5
6 # Insercion en heap: (f, g, nodo)
7 heapq.heappush(heap, (g[vecino] + h(vecino),
8     g[vecino], vecino))

```

Listing 2. Heurística haversine en A*

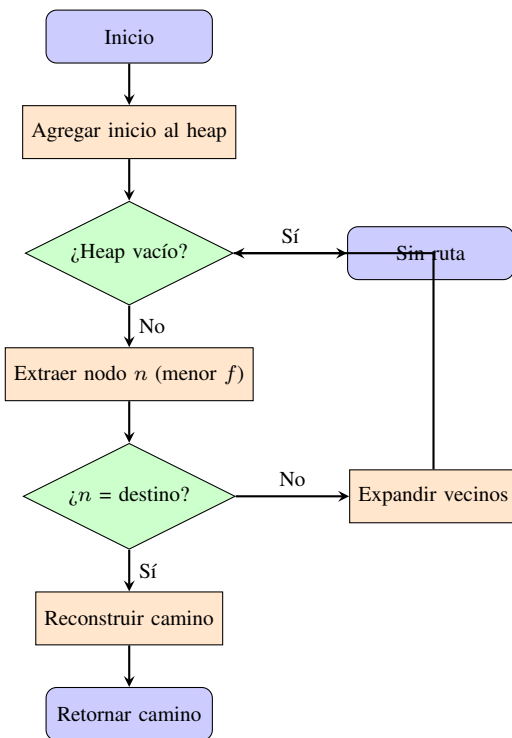


Figura 2. Diagrama de flujo del algoritmo A*.

III-C3. Diagrama de flujo de A*:

III-D. Interfaz Gráfica

La interfaz fue desarrollada con PyQt5 y se divide en dos zonas mediante un `QSplitter`: el panel de control (izquierda, 320 px fijo) y el mapa interactivo (derecha, expandible). El mapa utiliza la biblioteca Leaflet.js con teselas

de OpenStreetMap, embebido en un `QWebEngineView` con comunicación bidireccional vía `QWebChannel`.

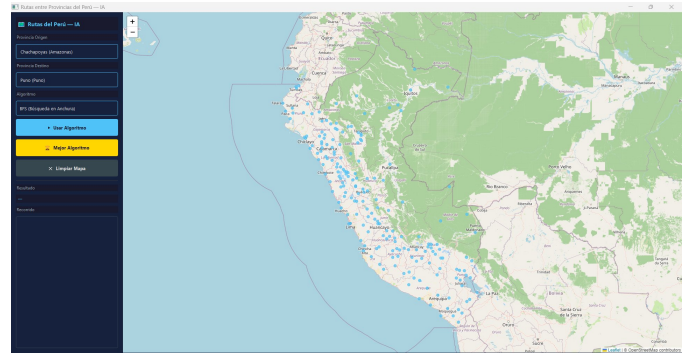


Figura 3. Pantalla principal del sistema mostrando las 196 provincias del Perú como nodos (marcadores azules) sobre OpenStreetMap.

El panel de control contiene:

- Combo editable de selección de provincia origen.
- Combo editable de selección de provincia destino.
- Selector del algoritmo a utilizar.
- Botón **Usar Algoritmo**: ejecuta el algoritmo seleccionado.
- Botón **Mejor Algoritmo**: ejecuta los 6 algoritmos A→B, los compara y muestra el ranking.
- Panel de resultado con distancia, pasos y tiempo.
- Lista de recorrido con el detalle de cada escala.

La ejecución de algoritmos se realiza en un `QThread` independiente para evitar bloqueos en la UI. La ruta resultante se dibuja sobre el mapa mediante JavaScript inyectado vía `runJavaScript()`.

IV. RESULTADOS

IV-A. Caso de Prueba: Lima → Puno

La ruta Lima–Puno (876 km en línea recta) es un caso representativo porque cruza varias regiones geográficas del Perú (costa, sierra central, altiplano). La Fig. 4 muestra el resultado de Dijkstra, y la Fig. 5 el ranking comparativo.

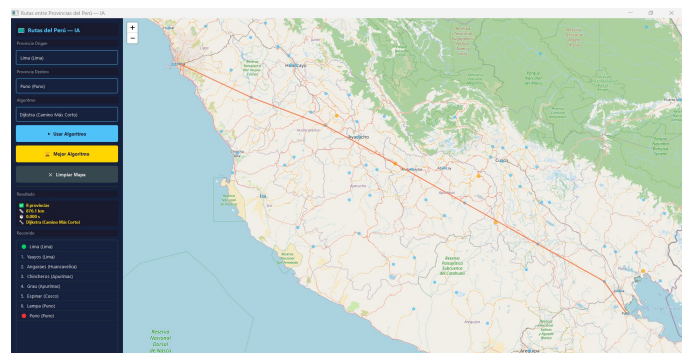


Figura 4. Resultado de Dijkstra para la ruta Lima → Puno: 8 provincias, 876.1 km, tiempo <1 ms.

La ruta encontrada por Dijkstra y A* es:

Lima → Yauyos → Angaraes → Chincheros → Grau → Espinar → Lamp

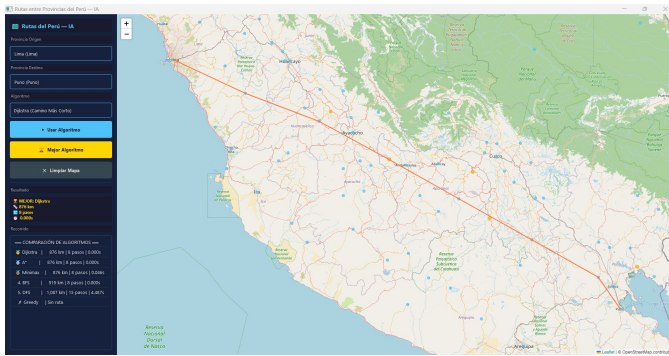


Figura 5. Ranking comparativo de los 6 algoritmos A→B para Lima → Puno. Dijkstra obtiene el primer lugar con 876 km.

IV-B. Comparación de Algoritmos

La Tabla II resume los resultados de los seis algoritmos sobre la ruta Lima–Puno:

Cuadro II
COMPARACIÓN DE ALGORITMOS — LIMA → PUNO

Algoritmo	Dist. (km)	Escalas	Tiempo	Óptimo
Dijkstra	876	8	<1 ms	Sí
A*	876	8	<1 ms	Sí
Minimax	876	8	48 ms	Sí*
BFS	919	8	<1 ms	No
DFS	1087	15	5.1 s	No
Greedy	—	—	<1 ms	No
TSP-SA	Recorre todas las provincias (objetivo distinto)			

*Minimax alcanza el óptimo en este caso por la heurística haversine. Greedy no encontró ruta: quedó atrapado en mínimo local.

IV-C. Análisis de Complejidad

Cuadro III
COMPLEJIDAD TEMPORAL Y ESPACIAL DE LOS ALGORITMOS

Algoritmo	Tiempo	Espacio
BFS	$O(V + E)$	$O(V)$
DFS	$O(V^d)$	$O(d)$
Dijkstra	$O((V + E) \log V)$	$O(V)$
A*	$O(E)$ mejor caso	$O(V)$
Greedy	$O(V^2)$	$O(V)$
Minimax	$O(b^d)$	$O(bd)$
TSP-SA	$O(n \cdot iter)$	$O(n)$

V: vértices, E: aristas, d: profundidad, b: branching factor

IV-D. Caso de Prueba: Arequipa → Tacna

Para validar en una ruta más corta (distancia directa ≈176 km):

- **Greedy:** Arequipa → Gral. Sánchez Cerro → Candarave, 176 km, 3 pasos.
- **Dijkstra:** misma ruta, 176 km.
- **BFS:** misma ruta, 176 km.

En rutas cortas Greedy funciona correctamente, confirmando que su limitación es para trayectos que requieren muchas escalas.

V. CONCLUSIONES

1. **Dijkstra y A*** son los algoritmos más eficientes para búsqueda de ruta más corta en este grafo: ambos garantizan optimalidad con tiempo de cómputo inferior a 1 ms para 196 nodos. A* añade la ventaja de explorar menos nodos gracias a la heurística haversine.
2. **El grafo de radio 150 km es la representación adecuada.** Un grafo completo (todas las provincias conectadas entre sí) hace que el camino directo A→B siempre tenga menor peso que cualquier ruta con escalas, eliminando el sentido del ejercicio. El radio de 150 km obliga a escalas reales con un promedio de 15.5 vecinos por nodo.
3. **BFS es eficiente en número de pasos pero no en distancia:** encuentra la ruta con menos escalas (8), pero no la más corta en kilómetros (919 vs. 876 de Dijkstra).
4. **DFS presenta las limitaciones esperadas:** sin poda basada en costo puede explorar caminos subóptimos (1087 km, 15 pasos) y su tiempo de ejecución es significativamente mayor (5 s).
5. **Greedy falla en rutas largas** por quedar atrapado en mínimos locales, pero es efectivo en rutas cortas (≤ 3 escalas). Esto ilustra de forma práctica la diferencia entre heurísticas locales y búsqueda global.
6. **Minimax, aunque diseñado para juegos adversariales,** puede adaptarse al problema de rutas modelando un adversario que intenta maximizar el costo. Con poda alfa-beta y la heurística haversine logra el resultado óptimo en este caso.
7. **TSP con Simulated Annealing resuelve un problema distinto:** no busca la ruta más corta entre dos puntos, sino el circuito que cubre todas las provincias con menor distancia total, siendo la opción adecuada para problemas de cobertura completa como rutas de distribución nacional.

AGRADECIMIENTOS

Al curso de Inteligencia Artificial de la Escuela Profesional de Ingeniería de Sistemas de la Universidad Nacional del Altiplano, Puno.

REFERENCIAS

- [1] S. Russell y P. Norvig, *Artificial Intelligence: A Modern Approach*, 3ra ed. Upper Saddle River, NJ: Prentice Hall, 2010.
- [2] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.
- [3] P. E. Hart, N. J. Nilsson y B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [4] C. E. Shannon, "Programming a computer for playing chess," *Philosophical Magazine*, vol. 41, no. 314, pp. 256–275, 1950.
- [5] W. J. Cook, *In Pursuit of the Traveling Salesman: Mathematics at the Limits of Computation*. Princeton, NJ: Princeton University Press, 2011.
- [6] S. Kirkpatrick, C. D. Gelatt y M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.